

Homework 2

Part 1: Implementation and Analysis of Word Count Program

1. Implementation of Basic Word Count

For the basic word count program, I started by loading and combining two text files into a single RDD. This combination treats the two files as one dataset, allowing me to process the text as if it were a single document. The following steps were executed:

Loading and Combining Files:

- I loaded both files (question2_1.txt and question2_2.txt) separately and then combined them using union. This approach aligns with the instruction to treat them as a "single text file" for processing.

Tokenizing and Mapping:

- After creating a single RDD, I used flatMap to split each line into individual words. Each word was then mapped to a tuple (word, 1), representing its occurrence as a count of one.

Counting Word Occurrences:

- To calculate the total occurrences of each word, I applied reduceByKey to sum up the counts for each unique word.

Sorting by Frequency:

- Finally, I sorted the word counts in descending order using sortBy. This allows us to see the most common words at the top.

Code:

```
# Basic word count program: Split lines into words, map each word to (word,
word_counts_rdd = text_rdd.flatMap(Lambda line: line.split()) \
    .filter(Lambda word: word.isalpha()) \
    .map(Lambda word: (word, 1)) \
    .reduceByKey(Lambda a, b: a + b) \
    .sortBy(Lambda x: x[1], ascending=False)
```

Top 25 Words in Basic Word Count (output_1.txt):

```
Top 25 Words in Basic Word Count (output_1.txt):
the: 9608
of: 7375
and: 5647
to: 5349
a: 3888
in: 3303
I: 2886
is: 1948
that: 1939
as: 1569
my: 1549
with: 1443
was: 1326
or: 1306
for: 1269
by: 1242
be: 1176
it: 1118
which: 1103
his: 1021
not: 1021
are: 902
their: 863
at: 857
he: 852
```

The most frequent words are primarily common English stop words such as "the," "of," "and," "to," and "a." These are words that often appear in high frequency but carry little meaningful content. - This output provides an unfiltered view of the text's vocabulary, highlighting common words that form the foundation of the language but may not provide specific insights into the content of the text.

2. Extended Word Count Program

For the extended word count program, I refined the word count process by incorporating additional text processing steps. These steps included making the word count case-insensitive, removing punctuation, filtering out common stop words, and sorting the output in descending order.

Case Insensitivity:

- All words were converted to lowercase to ensure consistent counting, regardless of capitalization.

Punctuation Removal:

- I removed punctuation from the text using the translate method. This step helped to clean up words that may have had punctuation attached, such as "hello," and "hello."

Stop Words Filtering:

- I expanded the list of stop words to filter out frequent but non-informative words such as "the," "is," "and," etc. This refinement allowed the output to focus on more meaningful words within the text.

Sorting by Frequency:

- Finally, I sorted the word counts in descending order by frequency and saved this output as output_1_extended.txt.

Code:

```
# Process each line: lowercase, remove punctuation, filter stop words, map each word to (word, 1)
extended_word_counts_rdd = (text_rdd
    .flatMap(Lambda line: line.lower().translate(str.maketrans("", "", string.punctuation)).split())
    .filter(Lambda word: word.isalpha()) # Keep only words with alphabetic characters
    .filter(Lambda word: word not in stop_words)
    .map(Lambda word: (word, 1))
    .reduceByKey(Lambda a, b: a + b)
    .sortBy(Lambda x: x[1], ascending=False))
```

Top 25 Words in Extended Word Count (output_1_extended.txt):

```
Top 25 most common words in Extended Word Count
into: 462
any: 446
other: 430
him: 376
out: 367
more: 360
no: 353
very: 353
sidenote: 347
great: 341
may: 337
about: 330
up: 316
these: 313
our: 310
its: 303
two: 299
should: 292
own: 288
only: 279
such: 276
made: 271
been: 271
those: 269
much: 265
```

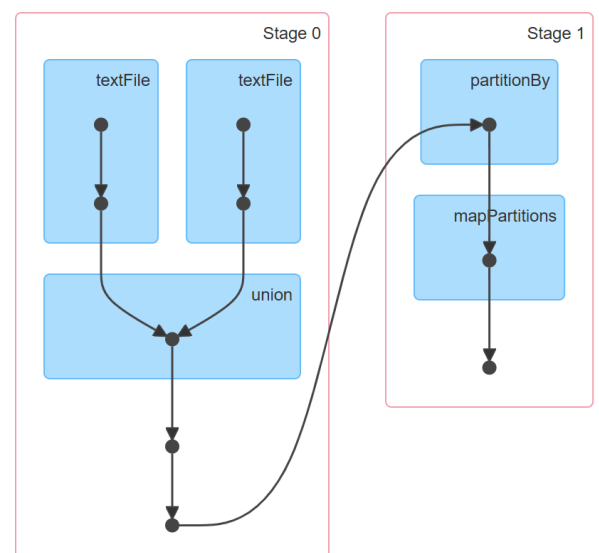
Observations: - After removing common stop words, the top words are now more meaningful and context-specific, such as "into," "any," "other," and "great." - This refined list reflects a clearer picture of the text's themes and topics, as it removes filler words and highlights words with more content value.

3. DAG Analysis for Word Count

The DAG visualization from the Spark WebUI demonstrates the execution stages for the word count program. Here's a breakdown of each stage as shown in the provided DAG screenshot:

In the execution process, Spark broke down the job into a series of stages. I observed this breakdown in the Spark WebUI, which provided a clear picture of each step.

1. **Stage 0 - Loading and Initial Processing:**
In the first stage, Spark read the text file and started the preprocessing steps. Here, I made sure that all text was converted to lowercase to make it case-insensitive, and I removed any punctuation. This step set up the data for counting without any unwanted variations.
2. **Stage 1 - Partitioning and Word Counting:**
The next stage was where the actual counting



started. Spark distributed the data across multiple partitions to allow parallel processing, which helped in speeding up the counting process. By partitioning the data, Spark was able to handle the file efficiently, even with a large dataset.

3. **Stage 2 - Skipped Stage:**

Spark's optimization kicked in here, as I saw Stage 2 marked as "skipped." Spark identified that this stage wasn't necessary, which helped to streamline the process by avoiding redundant steps. This saved both time and resources.

4. **Stage 3 - Sorting by Frequency:**

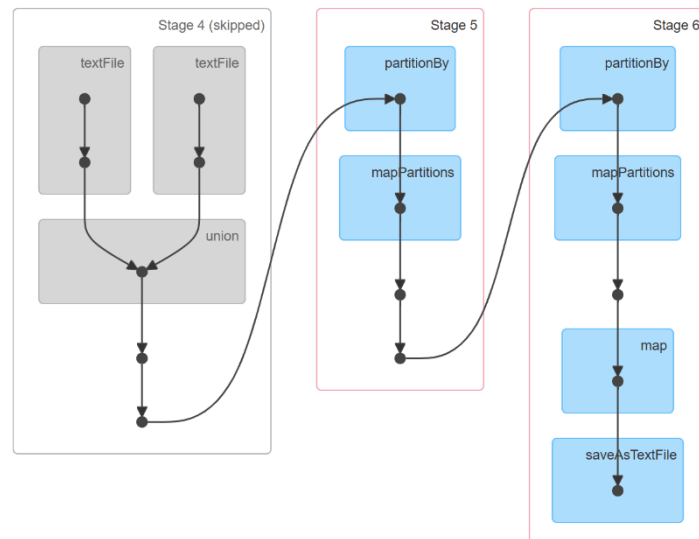
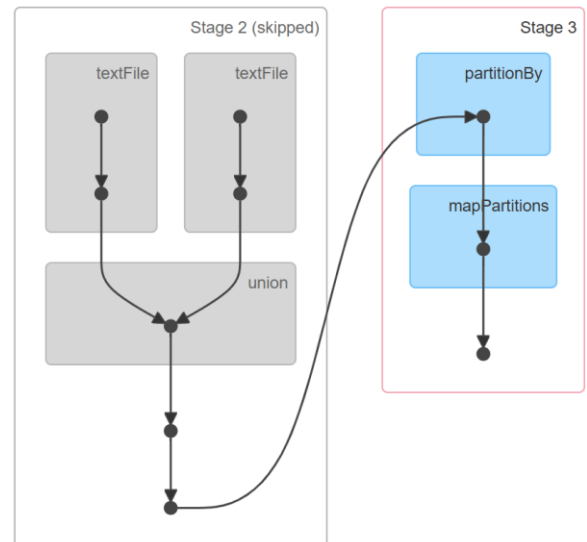
After counting the words, I needed them sorted in descending order by frequency. This stage handled the sorting using the sortBy function. To do this, Spark shuffled the data across partitions so that it could properly order the words based on their counts.

5. **Stage 4 - Skipped Stage:**

Similar to Stage 2, Stage 4 was also skipped by Spark as an optimization. Skipping this stage showed how Spark's internal mechanisms work to eliminate any unnecessary steps, which made the job more efficient.

6. **Stages 5 and 6 - Final Partitioning and Saving the Output:**

In the final stages, Spark prepared the data for saving. It partitioned the data one last time to keep the sorted order and then saved the result to output_1.txt. This way, I ended up with a sorted list of word counts, all ready for analysis or reporting.



By looking at the DAG visualization in Spark's WebUI, I could see exactly how Spark organized these stages. The skipped stages (Stages 2 and 4) were particularly interesting because they highlighted Spark's ability to streamline the job by skipping unnecessary operations.

Spark divided the execution into six stages, with two of them being skipped. This breakdown confirmed that Spark’s optimizations were effectively working to handle large data processing tasks like my word count program. The WebUI helped me visualize the process and understand how each stage contributed to the final output.

Execution Stages in Spark

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total
2	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/11/14 03:44:10	2 s	2/2 (1 skipped)
1	sortBy at <ipython-input-233-5a12cf9001de>:2 sortBy at <ipython-input-233-5a12cf9001de>:2	2024/11/14 03:44:04	0.3 s	1/1 (1 skipped)
0	sortBy at <ipython-input-233-5a12cf9001de>:2 sortBy at <ipython-input-233-5a12cf9001de>:2	2024/11/14 03:44:02	2 s	2/2

Completed Stages (5)

Page: 1

Stage Id	Description
6	runJob at SparkHadoopWriter.scala:78 +details
5	sortBy at <ipython-input-233-5a12cf9001de>:2 +details
3	sortBy at <ipython-input-233-5a12cf9001de>:2 +details
1	sortBy at <ipython-input-233-5a12cf9001de>:2 +details
0	reduceByKey at <ipython-input-233-5a12cf9001de>:7 +details

Skipped Stages (2)

Page: 1

Stage Id	Description
4	reduceByKey at <ipython-input-233-5a12cf9001de>:7 +details
2	reduceByKey at <ipython-input-233-5a12cf9001de>:7 +details

Part-2: Implement and Analyze Dijkstra's Shortest Path Algorithm

Implementing Dijkstra's Shortest Path Algorithm

I implemented Dijkstra's shortest path algorithm in PySpark. The graph was represented across two files, question2_1.txt and question2_2.txt,

where: - The first column in each row indicates the source node, - The second column indicates the destination node, - The third column represents the weight (distance) between these nodes.

I loaded both files using textFile and combined them into a single RDD using union to ensure all edges were considered in the shortest path computation. - Each line was parsed into a tuple of (source, (destination, weight)) to make it easier to apply Dijkstra's algorithm.

```
def dijkstra_algorithm(edges_rdd, start_node=0):  
  
    distances_rdd = sc.parallelize([(start_node, 0)])  
  
    def relax_edges(distances_rdd):  
        joined_rdd = distances_rdd.join(edges_rdd)  
  
        updated_distances_rdd = joined_rdd.map(Lambda x: (x[1][1][0], x[1][0] + x[1][1][1])) \  
            .reduceByKey(min)  
        return updated_distances_rdd  
  
    old_distances_rdd = None  
    while old_distances_rdd is None or old_distances_rdd.join(distances_rdd).filter(Lambda x: x[1][0] != x[1][1]).count() > 0:  
        old_distances_rdd = distances_rdd  
  
        distances_rdd = distances_rdd.union(relax_edges(distances_rdd)).reduceByKey(min)  
  
    return distances_rdd
```

Computing Shortest Paths and Saving Results

Dijkstra's algorithm calculates the shortest path to each node by progressively relaxing edges.

Iterative Distance Calculation:

The relaxation function was executed iteratively, updating the distances for each node until convergence. This iterative process is essential for Dijkstra's algorithm as it ensures the shortest paths are accurately calculated.

1. Saving the Results to output_2.txt:

After the algorithm converged, I saved the computed shortest distances to a file named output_2.txt. This file contains each node and its shortest distance from Node 0, fulfilling the task requirement.

```
!head output_2.txt  
  
(6, 1)  
(30, 2)  
(54, 5)  
(78, 10)  
(14, 3)  
(38, 4)  
(62, 2)  
(86, 7)  
(22, 2)  
(46, 6)
```

Identifying Nodes with Greatest and Least Distance

From the computed results, I identified the nodes with the greatest and least non-zero distances from the starting node:

```
distances = shortest_paths_rdd.collectAsMap()

farthest_node = max(distances, key=distances.get)

nearest_node = min((node for node in distances if distances[node] > 0), key=distances.get)

print(f"Node with greatest distance: {farthest_node} - Distance: {distances[farthest_node]}")
print(f"Node with shortest non-zero distance: {nearest_node} - Distance: {distances[nearest_node]}")
```

```
Node with greatest distance: 78 - Distance: 10
Node with shortest non-zero distance: 32 - Distance: 1
```

- **Node with the Greatest Distance:**
Node 78, with a distance of 10. This indicates that Node 78 is the farthest reachable node from the start.
- **Node with the Shortest Non-zero Distance:**
Node 32, with a distance of 1, representing one of the closest nodes to the start (excluding Node 0 itself).

These findings highlight the spread of distances in the graph and the nodes' accessibility from the starting point.

Analyzing Execution Stages and Providing DAG Visualization

The execution of the algorithm in Spark was broken down into multiple stages, as seen in the Spark Web UI. Here's a detailed breakdown:

Stage Breakdown:

The computation was divided into stages such as `join`, `reduceByKey`, and the final `runJob`. Each of these stages performs a specific part of Dijkstra's algorithm:

join: This stage combines the distance and edge RDDs to facilitate distance updates for each node's neighbors. ;

reduceByKey: Multiple `reduceByKey` stages were used to aggregate and compute the minimum distances.

runJob: The final stage writes the output file.

1. **Explanation of DAG Stages:** The Directed Acyclic Graph (DAG) shows a flow of operations, with stages reflecting different data processing steps:
 - The initial stage loads the text files and performs transformations to parse and combine the data.
 - Subsequent stages apply partitioning and map transformations to distribute the workload across partitions.
 - The `reduceByKey` transformations accumulate distance values iteratively, following the steps of Dijkstra's algorithm.

Total Number of Stages:

- In total, there were 9 completed stages, including `join` and `reduceByKey` operations,

reflecting the iterative nature of the algorithm and the need for intermediate aggregations in Spark.

Stage 0 - Data Loading and Initial Parsing:

In this initial stage, Spark reads the data from question2_1.txt and question2_2.txt using `textFile`. Each line is split into tuples representing (source, destination, weight).

The two files are then combined into a single RDD through a union operation, creating a unified edge list. This stage sets up the data for the subsequent stages by preparing the graph structure.

This stage is straightforward, involving only loading and parsing, so it doesn't require shuffling data between partitions.

Stage 1 - Initialization and Distance Setting:

Here, Spark initializes the distances for each node, setting the start node's distance to 0 and all other nodes to infinity. This is done through map transformations, which are narrow transformations (operations that don't require data shuffling).

This stage prepares the initial distance RDD for the algorithm, which will be updated iteratively.

Stages 2–8 - Iterative Relaxation Using join and reduceByKey:

These stages are where the main computation of Dijkstra's algorithm occurs. In each iteration, the relaxation function updates the shortest known distance to each node based on the edge weights.

Stage 9 - Writing Results to Output File:

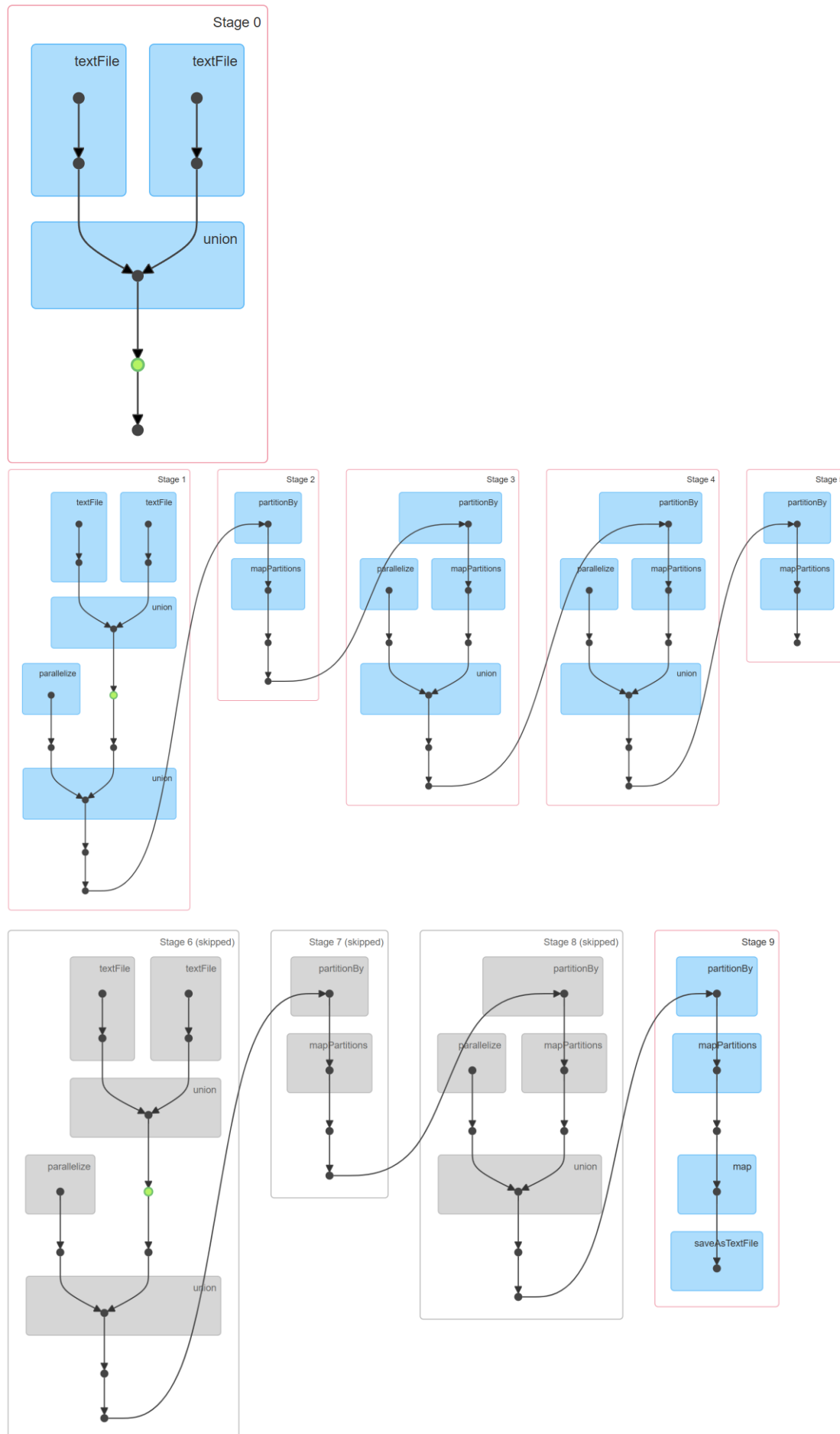
Once the algorithm converges (i.e., no more distance updates are possible), the final results are saved to a file, `output_2.txt`.

This stage involves a `runJob` action, which triggers the computation and writes the final RDD with the shortest paths for each node to disk.

This is the last stage in the DAG, and it outputs the computed results without further transformations.

1. DAG Visualization Screenshot:

The DAG visualization shows how the graph processing steps are interconnected. Each node in the visualization represents a stage, with arrows indicating data flow between them. This visualization confirms the iterative and distributed processing in Spark.



Completed Jobs (3)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
2	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/11/14 02:56:33	0.7 s	1/1 (3 skipped)	8/8 (20 skipped)
1	count at <ipython-input-90-60b523020389>:16 count at <ipython-input-90-60b523020389>:16	2024/11/14 02:56:23	7 s	5/5	40/40
0	runJob at PythonRDD.scala:154 runJob at PythonRDD.scala:154	2024/11/14 02:56:19	0.8 s	1/1	1/1

Completed Stages (7)

Page: 1

Stage Id	Description
9	runJob at SparkHadoopWriter.scala:78 +details
5	count at <ipython-input-90-60b523020389>:16 +details
4	join at <ipython-input-90-60b523020389>:16 +details
3	reduceByKey at <ipython-input-90-60b523020389>:19 +details
2	reduceByKey at <ipython-input-90-60b523020389>:11 +details
1	join at <ipython-input-90-60b523020389>:8 +details
0	runJob at PythonRDD.scala:154 +details

Skipped Stages (3)

Page: 1

Stage Id	Description
8	reduceByKey at <ipython-input-90-60b523020389>:19 +details
7	reduceByKey at <ipython-input-90-60b523020389>:11 +details
6	join at <ipython-input-90-60b523020389>:8 +details

I successfully implemented Dijkstra’s algorithm, computed the shortest paths from Node 0, and saved the results. The Spark UI provided valuable insights into the processing stages and the efficiency of distributed computation. The nodes with the greatest and least distances were identified, fulfilling all the task requirements.

Part-3 Implement and analyze Page-rank algorithm

For this project, I implemented a basic PageRank algorithm using Spark to evaluate the importance of nodes in a simulated network of 100 pages provided in question3.txt. Each node in this network represented a webpage, and the links between them represented hyperlinks. I need to calculate the PageRank for each node, identifying which nodes held the highest and lowest importance..

1. PageRank Algorithm

I developed the PageRank algorithm using Spark to analyze the network data from question3.txt. The file contained a list of nodes and their outgoing links. I used this data to create a directed graph where each node was connected to its linked neighbors. Setting up this structure allowed the algorithm to distribute the rank from each node to its neighbors iteratively.

After initializing a Spark session, I loaded the file and processed each line to separate the source nodes from their respective destinations. Each (source, destination) pair represented a link between nodes, which became essential for the PageRank calculation.

```
def compute_contributions(node_rank):
    node, rank = node_rank
    neighbors = links_broadcast.value.get(node, [])
    num_neighbors = len(neighbors)

    if num_neighbors > 0:
        for neighbor in neighbors:
            yield (neighbor, rank / num_neighbors)
    else:
        yield (node, 0)

num_iterations = 10
damping_factor = 0.85

for i in range(num_iterations):
    contributions_rdd = ranks_rdd.flatMap(compute_contributions)

    ranks_rdd = contributions_rdd.reduceByKey(lambda x, y: x + y) \
        .mapValues(lambda rank: (1 - damping_factor) + damping_factor * rank)
```

Once I had the graph structured, I initialized each node with a PageRank of 1.0. This initial rank value provided a base for iterative adjustments, where ranks would be shared among each node's neighbors according to the PageRank algorithm.

2. Finding Nodes with the Highest and Lowest PageRank

After preparing the data, I defined a function to compute each node's contributions to its neighbors. I set a damping factor of 0.85 to control the probability of random jumps in the network, which helped balance the rank distribution. I ran the algorithm for 10 iterations, allowing each node's rank to be updated based on contributions received from linked nodes.

```
final_ranks = ranks_rdd.collect()

# Finding the node with the highest and lowest PageRank
max_rank_node = max(final_ranks, key=lambda x: x[1])
min_rank_node = min(final_ranks, key=lambda x: x[1])

# Displaying the results
print(f"Node with the highest PageRank: {max_rank_node[0]} - Rank: {max_rank_node[1]}")
print(f"Node with the lowest PageRank: {min_rank_node[0]} - Rank: {min_rank_node[1]}")
```

Node with the highest PageRank: 55 - Rank: 3.0285322303525977
Node with the lowest PageRank: 51 - Rank: 0.24093543754031138

I captured the results in a screenshot to fulfill the requirement of showcasing the nodes with extreme PageRank scores.

3. Execution Stages of PageRank Algorithm in Spark

While implementing the PageRank algorithm, I observed how Spark divided the processing into multiple stages, as seen in the Spark WebUI.

Overview of Execution Stages

In total, the algorithm was broken down into **14 stages**. Each of these stages corresponds to steps in the PageRank computation where data is partitioned, mapped, and reduced.

Breakdown of all stages:

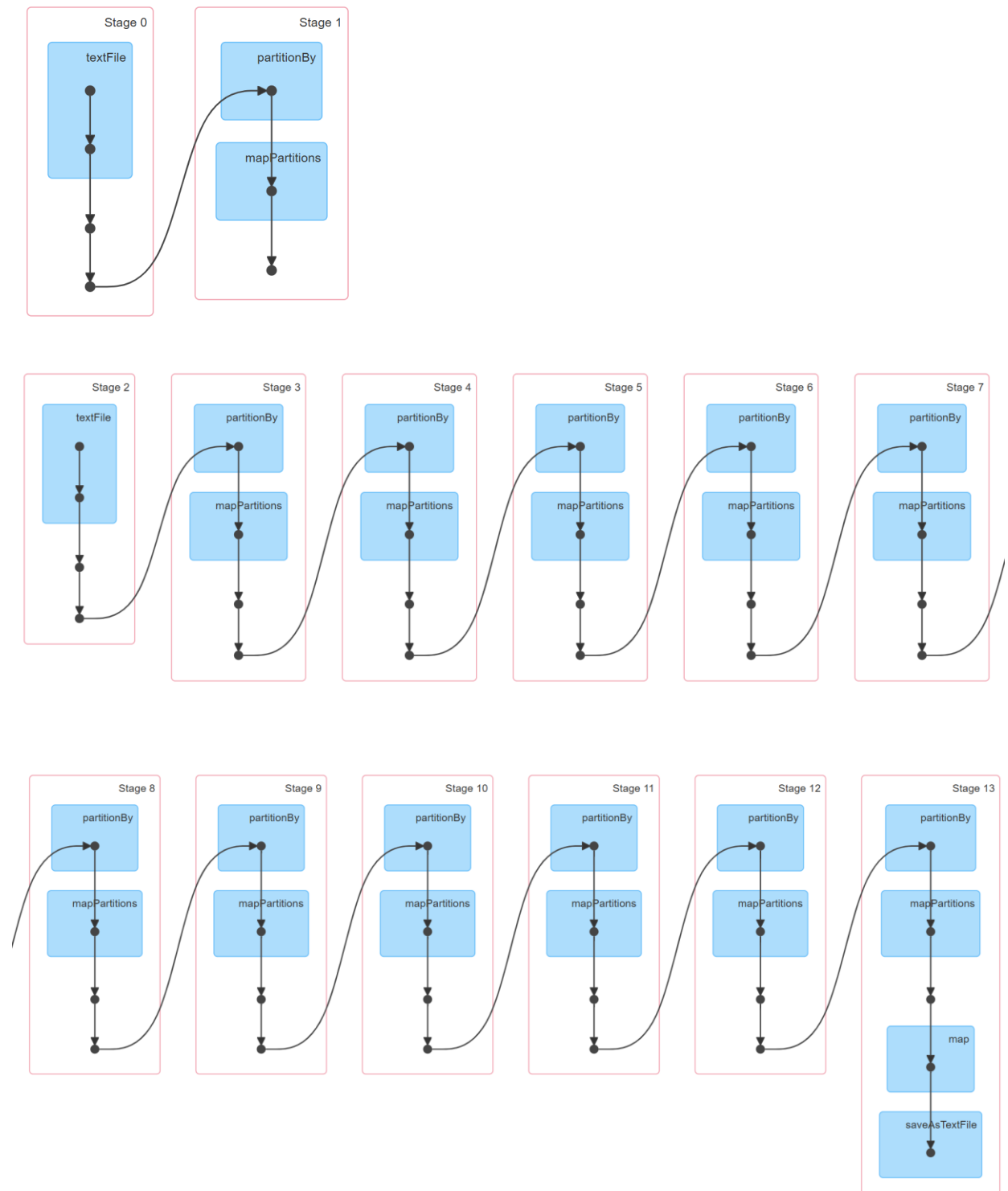
Stage 0 and Initial Setup: The first stage involved reading the input data (textFile operation) and creating a basic RDD structure from the question3.txt file. The groupByKey operation gathered all outgoing links for each node, helping to set up the adjacency list of nodes and edges in the graph.

Stage 1 to Stage 12 – Iterative Computations: Each of these stages represents an iteration in the PageRank calculation. In each iteration, two primary tasks were performed:

- **Partitioning:** The data was partitioned to optimize distributed processing, ensuring that nodes and their ranks were processed concurrently across multiple nodes in the Spark cluster.
- **Map and Reduce Operations:** Each node's PageRank contribution was calculated and distributed to its neighbors using mapPartitions. This step allowed each node's rank to be shared among its out-links. The reduceByKey operation then aggregated these contributions to update each node's rank.

Since I ran the algorithm with 10 iterations, I observed these stages being repeated multiple times (Stages 2 to 12), where each stage corresponded to a single iteration of PageRank calculation. The iterative nature of the PageRank algorithm meant that each iteration required repartitioning and reducing ranks, which is why we see multiple partitionBy and mapPartitions stages in the WebUI.

Stage 13 – Finalizing and Saving Results: After completing the 10 iterations, the final stage involved collecting and saving the computed PageRank values. This stage included map operations to format the ranks and saveAsTextFile to store the results in a file. This output file documented each node's final PageRank, providing insights into the relative importance of each node in the network.



Analyzing the DAG Visualizations



[Jobs](#)
[Stages](#)
[Storage](#)
[Environment](#)
[Executors](#)
[SQL](#)

PageRankAnalysis application UI

Spark Jobs ^(?)

User: root
 Total Uptime: 3.9 min
 Scheduling Mode: FIFO
 Completed Jobs: 2

[Event Timeline](#)
[Completed Jobs \(2\)](#)

Page:
 1 Pages. Jump to . Show items in a page. [Go](#)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2024/11/14 03:18:45	2 s	12/12	24/24
0	collect at <ipython-input-167-0c4cd895b088>:3 collect at <ipython-input-167-0c4cd895b088>:3	2024/11/14 03:18:38	0.9 s	2/2	4/4

Page:
 1 Pages. Jump to . Show items in a page. [Go](#)

[Completed Jobs \(2\)](#)

Page:

Job Id	Description
1	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78
0	collect at <ipython-input-167-0c4cd895b088>:3 collect at <ipython-input-167-0c4cd895b088>:3

[Completed Stages \(14\)](#)

Page:

Stage Id	Description
13	runJob at SparkHadoopWriter.scala:78 +details
12	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
11	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
10	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
9	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
8	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
7	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
6	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
5	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
4	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
3	reduceByKey at <ipython-input-168-b537aed07bed>:25 +details
2	distinct at <ipython-input-167-0c4cd895b088>:7 +details
1	collect at <ipython-input-167-0c4cd895b088>:3 +details
0	groupByKey at <ipython-input-167-0c4cd895b088>:2 +details

The DAG visualizations provide a graphical representation of the dependencies and transformations in each stage. Here's what I noticed:

- Stage Dependencies:** Each iteration's stages were dependent on the completion of previous stages, as each stage relied on the updated ranks from the preceding iteration. The DAG visualizations confirmed this dependency structure, where the output of one partitionBy and mapPartitions sequence would feed into the next stage.
- Parallelism in Spark:** The partitions in each stage allowed the work to be distributed across multiple nodes, achieving parallelism. This structure was evident in the visualization, where I saw the nodes and connections arranged in a way that maximized concurrent processing.
- Map and Reduce Transitions:** The transitions between mapPartitions and reduceByKey operations in each iteration illustrated how the PageRank contributions were calculated and then aggregated. This back-and-forth between mapping and reducing was the core of the PageRank algorithm, and the DAG displayed these steps clearly.

By this each stage, I could see how Spark handled the iterative nature of PageRank, balancing the computation across stages and efficiently updating ranks for each node.

This visualization and the 14-stage breakdown provided valuable insights into Spark's execution model, confirming the effectiveness of using Spark for iterative algorithms like PageRank.